מספר תעודת זהות
יש לרשום את כל תשע הספרות

יש להדביק כאן את מדבקת הנבחן

האוניברסיטה הפתוחה

י' בתמוז תשפ"ד

16
ביולי 2024

סמסטר 2024ב

מס' שאלון - 525

מס' מועד 87

4 / 20905

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה **12** עמודים

מבנה הבחינה:

בבחינה 3 שאלות.
משך זמן הבחינה 3 שעות.
עליכם לענות על כל השאלות.
משקל השאלות מפורט בגוף השאלון.

שימו לב:
את תשובותיכם רשמו אך ורק במחברת הבחינה.
כתבו בכתב יד ברור ומסודר.

חומר עזר המותר בשימוש:
ספר הקורס "Essentials of Programming Languages".
מדריך הלמידה "שפות תכנות".

חומר עזר:
ספר הקורס: essentials of programming languages
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.
אסור שימוש בעזרים ובחומרים מקוונים.

בהצלחה !!!

אינכם חייבים
להחזיר את השאלון לאוניברסיטה הפתוחה

שאלה 1 (35 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81.

המצב הקיים בשפה הוא שניתן להגדיר פרוצדורה בעלת פרמטר יחיד (ביטוי proc-exp) וכן ניתן להפעיל פרוצדורה כזו (ביטוי call-exp).
בשאלה זו נרצה לשנות את האופן שבו פרוצדורה מוגדרת ומופעלת, ולכן הביטוי proc-exp יבוטל ויוחלף בביטוי חדש בשם overload-proc-exp. והביטוי call-exp ישתנה בהתאם.
לאחר השינויים תתאפשר הגדרת פרוצדורה עם אפשרות להעמסה (Overloading) כלומר, יכולות להיות לפרוצדורה מספר גרסאות. ובהפעת פרוצדורה תופעל הגרסה המתאימה (אם קיימת כזו)

להלן הדקדוקים החדשים המגדירים את אופן הגדרת פרוצדורה והפעלתה:

$$\textit{Expression} ::= \text{overload proc } \{ \; [\; \{\textit{identifier}\}^{*(,)} \;] \; \text{->} \; \textit{Expression} \; ; \}^{+}$$

overload-proc-exp (ids, bodies)

הסימן <- נכתב ע"י התו מקף (-) ולאחריו התו של סימן גדול מ- (<)

$$\textit{Expression} ::= (\textit{Expression} \; \{ \; \textit{Expression} \; \}^{*} \;)$$

call-exp (rator, rands)

הסבר כללי:

בביטוי של הגדרת פרוצדורה, ניתן להגדיר לפרוצדורה מספר גרסאות, לכל גרסה יש כמות פרמטרים שונה וגוף פרוצדורה שונה לביצוע. בביטוי של הגדרת פרוצדורה חייבת להופיע לפחות גרסה אחת, אחרת תודפס שגיאה מתאימה.

בביטוי של הפעלת פרוצדורה, יכולים להישלח מספר פרמטרים, ולכן יש להפעיל את הגרסה המתאימה של הפרוצדורה בהתאם לכמות הפרמטרים. אם אין גרסה כזו, יש להדפיס הודעת שגיאה מתאימה.

להלן מספר דוגמאות להמחשת השימוש בביטויים החדשים.

דוגמאות להמחשת אופן השימוש במרכיבים החדשים:

דוגמה 1:

```
> (run " let p = overload proc
                              [a,b] -> -(a,b);
                              [] -> 90;
                              [x] -> -(x,1);
                 in
                     (p 30 40 )")
(num-val -10)
```

הסבר דוגמה 1:

בדוגמה 1, מוגדרת פרוצדורה בשם p שלה מספר גרסאות. כל גרסה כוללת כמות פרמטרים שונה וגוף פרוצדורה שונה לביצוע. הגרסה הראשונה כוללת 2 פרמטרים, הגרסה השניה היא כפרוצדורה ללא פרמטרים, והגרסה השלישית היא של פרוצדורה בעלת פרמטר יחיד. בגוף פעולת ה-let מופעלת הפרוצדורה p עם 2 פרמטרים, וזה מתאים לגרסה הראשונה שלה, ולכן מבוצעת פעולת החיסור בין הפרמטרים.

דוגמה 2:

```
> (run " let p = overload proc
                              [a,b] -> -(a,b);
                              [] -> 90;
                              [x] -> -(x,1);
                 in
                     (p )")
(num-val 90)
```

הסבר דוגמה 2:

בדוגמה 2, מוגדרת פרוצדורה בשם p שלה מספר גרסאות (כמו בדוגמה הקודמת). בגוף פעולת ה-let מופעלת הפרוצדורה p ללא פרמטרים, וזה מתאים לגרסה השניה שלה, ולכן מוחזר 90.

דוגמה 3:

```
> (run " let p = overload proc
                                [a,b] -> -(a,b);
                                [] -> 90;
                                [x] -> -(x,1);
                  in
                      (p 5)")
(num-val 4)
```

הסבר דוגמה 3:

בדוגמה 3, מוגדרת פרוצדורה בשם p שלה מספר גרסאות (כמו בדוגמה הקודמת). בגוף פעולת ה-let מופעלת הפרוצדורה p עם פרמטר יחיד שערכו 5, וזה מתאים לגרסה השלישית שלה, ולכן מוחזר 4.

דוגמה 4:

```
> (run " let p = overload proc
         in
          (p 30 40 70)")
interp.scm:81:29: proc-definition: You must define at least one procedure
>
```

הסבר דוגמה 4:

בדוגמה 4, מוגדרת פרוצדורה בשם p שלה אפס גרסאות (ועל פי הדקדוק הנתון זה אינו תקין). ולכן התקבלה הודעת שגיאה מתאימה.

דוגמה 5:

```
> (run " let p = overload proc
                          [a,b] -> -(a,b);
                          [] -> 90;
                          [x] -> -(x,1);
                in
                   (p 30 40 70)")
interp.scm:98:15: Calling-proc: No procedure with 3 arguments found
```

הסבר דוגמה 5:

בדוגמה 5, מוגדרת פרוצדורה בשם p שלה מספר גרסאות. בגוף פעולת ה-let מופעלת הפרוצדורה p עם 3 פרמטרים, ומאחר ואין גרסה של הפרוצדורה עם כמות כזו של פרמטרים, התקבלה הודעת שגיאה מתאימה.

<u>ממשו והטמיעו</u> את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת **<u>"וישגור"</u>** (**<u>שפת PROC</u>**). בתשובתכם, הקפידו להסביר **<u>היכן בדיוק</u>** נדרשים שינויים ותוספות בקבצי המפרש, **<u>ומהם</u>** השינויים והתוספות.

<u>הנחיות כלליות לפתרון:</u>

1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
3) הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier

ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור ($\{\ \}^*$ או $\{\ \}^+$)

שאלה 2 (35 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "**וירמוז**"(IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטויים חדשים המאפשרים עבודה עם אירועים (Events)

אירוע (Event), מאפשר לרשום אליו פרוצדורות אשר יופעלו כאשר האירוע מתרחש (Invoke)

בשאלה זו, נדרש להוסיף יכולת ליצירת אירוע חדש, רישום פרוצדורה/ות לאירוע, והפעלת אירוע.
לשם כך נוסיף לשפה 2 ביטויים חדשים, אחד המאפשר יצירת אירוע חדש, השני מאפשר הוספת פרוצדורות לאירוע קיים.
כמו כן, נשנה את מימוש הביטוי הקיים בשפה להפעלת פרוצדורה, כך שיתאים **הן להפעלת פרוצדורה והן להפעלת אירוע.** התחביר של ביטוי הפעלה לא משתנה אלא רק התנהגותו.

להלן הדקדוקים המגדירים את הביטויים שנדרש לממש :

Expression **::=** event.create()

event-create-exp( )

Expression **::=** <*Identifier*> += { { *Expression* }$^{+(;)}$ }

event-add-exp(evnt , procs)

הסבר על פעולת הביטויים החדשים :

- ביטוי **event-create-exp** אמור ליצור ולהחזיר אירוע (event) חדש. (שימו לב, שיש כאן טיפוס נתונים חדש)

- ביטוי **event-add-exp** מאפשר להוסיף לאירוע evnt שכבר נוצר, אוסף של לפחות פרוצדורה אחת.

- ביטוי **call-exp** הקיים בשפה ונועד להפעלת פרוצדורה, הוא נשאר עם אותו התחביר, אך מימושו צריך להתאים הן להפעלת פרוצדורה, והן להפעלת אירוע. בהפעלת אירוע מופעלות כל הפרוצדורות שנוספו (נרשמו) לאירוע, הפרוצדורות מופעלות לפי הסדר בו נרשמו. לכל אחת מהפרוצדורות המופעלות נשלח הפרמטר שצויין בביטוי ההפעלה.

נדרש לבדוק ולהדפיס הודעות שגיאה מתאימות כתוצאה משימוש שגוי באירועים.

להלן מספר דוגמאות (תקינות וגם לא תקינות) להמחשת הנדרש:

דוגמה 1:

```
> (run  "
          let a=1000
          in
           let dec_x= proc (x) set a= -(a,x)
           in
             let OnClick = event.create()
             in
               begin
                <OnClick> += {dec_x;dec_x;dec_x;dec_x};
                (dec_x 100);
                (OnClick 50);
                a
                end")


(num-val 700)
>
```

בדוגמה 1 מוגדר משתנה a שערכו 1000, ופרוצדורה בשם dec_x. לאחר מכן מוגדר אירוע בשם OnClick, ובהמשך מתווספת (נרשמת) אליו הפרוצדורה dec_x 4 פעמים.
לאחר מכן, מופעלת הפרוצדורה dec_x עם פרמטר 100, דבר שגורם למשתנה a להשתנות ל- 900. הפעלה זו באה להדגים שביטוי הפעלה עדיין תומך בהפעלת פרוצדורה.
לאחר מכן, מופעל האירוע OnClick עם פרמטר 50, דבר שגורם להפעלת כל הפרוצדורות שנרשמו לאירוע לפעול עם פרמטר 50, התוצאה של כך במקרה זה, היא הקטנת המשתנה a ב-200 (כל פעם הוא קטן ב- 50 בעקבות הפעלת 4 הפרוצדורות שנרשמו לאירוע). לכן ערכו של a הוא סה"כ 700. לבסוף מוחזר ערכו של a.
בדוגמה זו ניתן לראות הדגמה של אופן הגדרת אירוע, אופן רישום פרוצדורות לאירוע, ואופן הפעלת אירוע או פרוצדורה.

דוגמה 2:

```
> (run "
        let a=10
        in
          let b=200
          in
            let OnClick = event.create()
            in
              let dec_x = proc (x) set a = -(a,x)
              in
                let add_x = proc(x) set b = -(b , -(0,x))
                in
                  begin
                     <OnClick> += {dec_x; proc (y) set a= -(b,y);  add_x };
                     (OnClick 20);
                     -(a,b)
                  end")
(num-val -40)
>
```

בדוגמה 2 מוגדרים 2 משתנים a ו- b , 2 פרוצדורות dec_x ו- add_x, וכן אירוע בשם OnClick לאירוע נרשמות הפרוצדורה dec_x, פרוצדורה אנונימית, והפרוצדורה add_x.
לאחר מכן, מופעל האירוע OnClick עם פרמטר 20, בהפעלת האירוע מופעלת תחילה הפרוצדורה dec_x שמשנה את ערכו של a ל- 10-, לאחר מכן, מופעלת הפרוצדורה האנונימית שמשנה את ערכו של a ל- 180. לאחר מכן מופעלת הפרוצדורה add_x המשנה את ערכו של המשתנה b ל- 220.
לבסוף התוכנית מחזירה את ההפרש בין a ל- b ולכן תוצאתה 40-

דוגמה 3:

```
> (run  "
        let a=10
        in
         let OnClick = event.create()
            in
              <OnClick> += {}

                  ")
```

```
interp.scm:112:27: event-add-exp: an empty list is not allowed
>
```

בדוגמה 3 מודגמת הגדרת אירוע, ורישום אליו עם רשימת פרוצדורות ריקה (לא תקין תחבירית) על כן מודפסת הודעת שגיאה מתאימה.

דוגמה 4:

```
> (run  "
          let a=10
          in
           let p= proc (x) -(x,5)
           in
             let OnClick = event.create()
             in
                <a> += {p}")
```

```
interp.scm:118:23: event-add-exp: a is not an event
> |
```

בדוגמה 4 מודגם ניסיון לרשום פרוצדורות ל-identifier שאינו מייצג אירוע, על כן מודפסת הודעת שגיאה מתאימה.

דוגמה 5:

```
> (run  "
         let a=10
         in
          let p= proc (x) -(x,5)
          in
            let OnClick = event.create()
            in
               <OnClick> += {p;7}")
```

```
interp.scm:156:12: event-add-exp: event cannot contain non-procedure elements
> |
```

בדוגמה 5 מודגם ניסיון לרשום לאירוע ביטוי שאינו מייצג פרוצדורה, על כן מודפסת הודעת שגיאה מתאימה.

דוגמה 6:

```
> (run  "
          let a=10
          in
           let p= proc (x) -(x,5)
           in
             let OnClick = event.create()
             in
                (OnClick 20)")
```

```
interp.scm:97:35: event-call: cannot invoke an empty event
> |
```

בדוגמה 6 מודגם ניסיון להפעיל אירוע ריק (אירוע שהוגדר אך לא נרשמו אליו פרוצדורות), על כן מודפסת הודעת שגיאה מתאימה

דוגמה 7:

```
> (run  "
         let a=1000
         in
          let dec_x= proc (x) set a= -(a,x)
          in
            let OnClick = event.create()
            in
              begin
               <OnClick> += {dec_x;dec_x};
               (a 100)
               end")
```

```
interp.scm:99:20: call-exp: only procedure or event can be invoked
>
```

בדוגמה 7 מודגם ניסיון להפעיל ביטוי שאינו מייצג פרוצדורה או אירוע, על כן מודפסת הודעת שגיאה מתאימה

דוגמה 8:

```
> (run  "
          let a=1000
          in
           let dec_x= proc (x) set a= -(a,x)
           in
             let OnClick = event.create()
             in
               let OnOpen = event.create()
               in
               begin
                 <OnClick> += {dec_x;dec_x;dec_x;dec_x};
                 <OnOpen> +=  {proc (x) if zero?(x) then set a=350 else set a= 550};
                 (OnClick 50);
                 set OnClick = OnOpen;
                 (OnClick 0);
                 a
               end")


(num-val 350)
>
```

בדוגמה 8 מודגמת עבודה עם 2 אירועים שונים. מוגדרים האירועים OnClick ו- OnOpen, בתחילה מבוצע רישום פרוצדורות לאירועים אלה, לאחר מכן, מופעל האירוע OnClick עם פרמטר 50, דבר הגורם למשתנה a להשתנות לערך של 800. לאחר מכן, מבוצעת פעולת השמה המשנה את האירוע OnClick להיות האירוע OnOpen, ושוב מופעל האירוע OnClick שהפעם משנה את ערכו של a ל-350. ולבסוף מוחזר ערכו של a. לכן תוצאת הרצת התוכנית היא 350.

ממשו והטמיעו את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת **"וירמוז"** (**שפת Implicit-Refs**).

בתשובתכם, הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות.

הנחיות כלליות לפתרון:

6) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
7) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
8) הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
9) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
10) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
11) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור ($\{\ \}^*$ או $\{\ \}^+$)

שאלה 3 (30 נקודות)

להלן תוכנית בשפת PROC (שפת "וישגור")

```
> (run "
    (let x =proc (x)   (x 6)
     in  x  proc (x)  -(x,1)
    )
")
```

תוצאת הריצה של תוכנית זו היא :

א) proc-val

ב) (num-val 6)

ג) (num-val 5)

ד) (bool-val #t)

ה) (bool-val #f)

ו) התוכנית מחזירה שגיאה בזמן הריצה.

נמקו והסבירו את תשובתכם ע"י הסבר של דרך חישוב תוצאת התוכנית

את תשובתכם רשמו במחברת הבחינה בלבד.

ב ה צ ל ח ה !